REPORT

DESIGN AND ANALYSIS OF ALGORITHMS

Title

Evaluation of Quick Sort Performance on Extremely Large Datasets Using Sampling Techniques for Pivot Selection

---

Aim

To evaluate the performance of Quick Sort on a large dataset by comparing a conventional pivot selection technique with a sampling-based pivot selection technique, measure their execution times, and analyze how sampling affects pivot quality and sorting performance.

Objective

The main objectives are:

1. Implement Quick Sort.
2. Generate a large dataset.
3. Measure Quick Sort execution time.
4. Use sampling for pivot selection.
5. Compare normal and sampling-based Quick Sort.
6. Analyze the quality of the selected pivots.
7. Determine whether sampling improves performance.

Introduction

Quick Sort is a divide-and-conquer sorting algorithm. It selects a pivot and partitions the array into two parts:

- Elements smaller than or equal to the pivot.
- Elements greater than the pivot.

The process is repeated recursively until the entire array is sorted.

The performance of Quick Sort strongly depends on **pivot selection**.

A poor pivot can produce highly unbalanced partitions, while a good pivot produces partitions that are closer to equal size.

Standard Quick Sort

In the standard implementation, the last element of the current subarray is selected as the pivot.

For example:

```
[8, 3, 7, 2, 9, 5]
               ↑
            Pivot
```

The array is partitioned around the pivot.

However, selecting a fixed position can sometimes result in poor partitions, especially for certain input arrangements.

Sampling-Based Pivot Selection

To improve pivot quality, sampling can be used.

In this implementation, **three elements** are sampled:

```
First element
Middle element
Last element
```

The median of these three values is selected as the pivot.

For example:

```
First = 10
Middle = 4
Last = 7
```

The values are:

Therefore, the median is:

So `7` is selected as the pivot.

Why Sampling Helps

A pivot near the middle of the value distribution tends to create more balanced partitions.

For example:

Poor pivot

```
100 elements

Pivot
 |
 v
[99 elements] [0 elements]
```

This produces highly unbalanced recursion.

Better pivot

```
100 elements

Pivot
 |
 v
[50 elements] [49 elements]
```

This produces more balanced recursion.

Sampling increases the probability of selecting a pivot closer to the center of the data.

Algorithm

Normal Quick Sort

1. Select the last element as pivot.
2. Partition the array around the pivot.
3. Recursively sort the left partition.
4. Recursively sort the right partition.

5. Continue until the partitions contain at most one element.

Sampling Quick Sort

1. Select three sample elements.
2. Find their median.
3. Use the median as the pivot.
4. Partition the array.
5. Recursively sort the two partitions.
6. Continue until the array is sorted.

Dataset

The program generates a large dataset containing:

The **same dataset** is used for both algorithms to make the comparison fair.

Experimental Method

The experiment follows these steps:

1. Generate 1,000,000 random integers.
2. Make two copies of the same dataset.
3. Sort the first copy using normal Quick Sort.
4. Record its execution time.
5. Sort the second copy using sampling-based Quick Sort.
6. Record its execution time.
7. Verify that both arrays are sorted correctly.
8. Compare the measured execution times.

Source Code

The implementation uses:

- C programming language
- `clock()` for execution-time measurement
- Random data generation

- Standard Quick Sort
- Median-of-three pivot sampling

Source file:

`QuickSort_Sampling.c`

Expected Output Format

Your actual output will depend on your computer.

```
Quick Sort Performance Evaluation
==================================
Dataset size: 1000000 elements

Normal Quick Sort completed.
Sorted correctly: Yes
Execution Time: X.XXXXXX seconds

Sampling Quick Sort completed.
Sorted correctly: Yes
Execution Time: Y.YYYYYY seconds

Performance Comparison
----------------------
Normal Quick Sort   : X.XXXXXX seconds
Sampling Quick Sort : Y.YYYYYY seconds
```

Take a screenshot of your actual output and add it here.

Time Complexity Analysis

Let be the number of elements.

Best/Average Case

When the pivot divides the array approximately equally:

Therefore:

Worst Case

If the pivot repeatedly produces extremely unbalanced partitions:

Therefore:

Effect of Sampling

Sampling does **not change the theoretical worst-case bound** of Quick Sort.

The worst case can still be:

However, sampling generally improves the **probability of selecting a good pivot**, which can reduce the occurrence of highly unbalanced partitions.

Therefore, sampling can improve practical performance, especially for large datasets.

Performance Comparison

Feature	Normal Quick Sort	Sampling Quick Sort
Pivot selection	Fixed position	Median of sample
Partition quality	Can vary	Usually more balanced
Average complexity	O(n log n)	O(n log n)
Worst-case complexity	$O(n^2)$	$O(n^2)$
Pivot selection overhead	Low	Slightly higher
Large dataset performance	Good	Often more robust

Analysis of Results

The measured execution times should be compared using the output obtained from the program.

If the sampling-based implementation produces a lower execution time, it indicates that the additional pivot-selection work was compensated for by better partitioning.

If the normal implementation is faster in a particular run, this does **not** mean sampling is ineffective. The benefit depends on the input distribution, machine, compiler, and dataset.

For a more reliable experiment, the program can be executed multiple times and the average execution time can be reported.

Advantages of Sampling

- Improves pivot selection.
- Reduces the probability of extremely unbalanced partitions.
- Can improve practical Quick Sort performance.
- Particularly useful when input order may be unfavorable.
- Adds only a small amount of pivot-selection work.

Limitations

- Sampling adds extra comparisons.
- It does not eliminate the theoretical worst case.
- Performance improvement depends on the input data.
- Execution time can vary between runs and systems.

Applications

Sampling-based Quick Sort can be useful when sorting:

- Large research datasets
- Database records
- Log files
- Numerical datasets
- Search engine data
- Large-scale analytics data

Result

Quick Sort was evaluated on a large dataset using two pivot-selection strategies: a conventional fixed pivot and **median-of-three sampling**.

The experiment demonstrates that pivot quality has a significant effect on Quick Sort performance. Sampling requires a small amount of additional work but can produce more balanced partitions and reduce the likelihood of poor recursive splits.

Therefore, sampling is an effective practical technique for improving Quick Sort robustness on large datasets, while the theoretical worst-case complexity remains:

and the expected average-case performance remains: